

Phase 3: ALU + Sequencer Integration

Objective

Combine the validated DINO sequencer with the standalone ALU to create a programmable system capable of both computational and display operations.

Component BOM

Combines Phase 1 + Phase 2 components plus integration logic

Additional Components

Qty	Component	Function
1	MC74F521N	8-bit comparator (instruction decode)
2	74F08	Quad AND gates (control logic)
1	74F02	Quad NOR gates (control logic)
1	74F32DC	Quad OR gates (signal combining)
1	SN74ALS573BN	Transparent latch (instruction register)
2	74F244N	Octal buffers (bus isolation)
4	0.1µF capacitors	Additional decoupling
8	Toggle switches	Manual instruction mode select
-	Additional breadboard space	Integration wiring

Existing Components (from Phases 1 & 2)

- DINO v0.0.1 complete system
- Standalone ALU system
- Power supplies and test equipment

System Architecture

Memory Map

- 0x0000-0x7FFF: Program instructions
- 0x8000-0x8FFF: Data storage area (future use)
- 0x9000-0x9FFF: I/O space (future peripherals)

Instruction Format (8-bit)

Bits [7:4] = Opcode (16 possible instructions)
Bits [3:0] = Operand/immediate data/register select

Initial Instruction Set

Opcode	Mnemonic	Description
0000	NOP	No operation, display current data
0001	LOAD_A	Load immediate into Register A
0010	LOAD_B	Load immediate into Register B
0011	ADD	A + B → Output latch
0100	SUB	A - B → Output latch
0101	AND	A AND B → Output latch
0110	OR	A OR B → Output latch
0111	XOR	A XOR B → Output latch
1000	DISPLAY	Show immediate value on LEDs
1001-1111	Reserved	Future expansion

Circuit Integration

Bus Architecture Enhancement

DINO Sequencer:
Counter → SRAM Address
SRAM Data → Instruction Register (SN74ALS573BN)

Instruction Decode:
Instruction Register → MC74F521N comparator
Comparator outputs → Control logic (74F08, 74F02, 74F32DC)

Data Routing:
Control signals → Bus multiplexers
Route data to: ALU registers OR LED display

Control Signal Generation

Instruction Decode Logic:

LOAD_A_EN = (Opcode == 0001)

LOAD_B_EN = (Opcode == 0010)

ALU_OP_EN = (Opcode >= 0011) AND (Opcode <= 0111)

DISPLAY_EN = (Opcode == 1000) OR (Opcode == 0000)

Data Path Control

Normal Display Mode:

SRAM Data → 74F244 buffer → LEDs (DINO behavior)

ALU Operations:

SRAM Data → Register A or B (based on instruction)

ALU Result → Output latch → LEDs

Mixed Mode:

Instruction determines routing via control signals

Build Steps

1. Instruction Register Addition

- Add SN74ALS573BN latch between SRAM data output and instruction decode
- Connect latch enable to system clock
- Verify instruction data captured correctly

2. Instruction Decode Logic

- Wire MC74F521N to compare instruction bits [7:4] against known opcodes
- Use 74F08/74F02/74F32DC to generate control signals
- Test with manual instruction input before automation

3. Data Path Switching

- Add 74F244 buffers to isolate different data paths
- Connect control signals to buffer enable pins
- Verify only one data path active at a time

4. ALU Integration

- Connect SRAM data bus to ALU register inputs via buffers

- Route ALU output through existing output latch
- Add control logic for ALU function selection

Test Procedures

1. Instruction Fetch Test

Objective: Verify instructions load correctly from memory

Procedure:

1. Program SRAM with known instruction sequence
2. Step through addresses manually
3. Verify instruction register captures each byte
4. Check instruction decode outputs with logic analyzer

Test Data:

Address	Data	Expected Decode
0x0000	0x10	LOAD_A opcode detected
0x0001	0x20	LOAD_B opcode detected
0x0002	0x30	ADD opcode detected
0x0003	0x80	DISPLAY opcode detected

2. Data Path Isolation Test

Objective: Verify bus contention prevention

Procedure:

1. Set instruction to LOAD_A (0x1F)
2. Verify only ALU Register A receives data
3. Confirm LEDs and Register B are isolated
4. Repeat for each instruction type

Success Criteria: No bus contention, single data path active per instruction

3. Simple Program Execution Test

Objective: Execute a basic computational program

Test Program:

Address	Instruction	Data	Description
-----	-----	-----	-----
0x0000	0x1F	-	Load 0xF into Register A
0x0001	0x21	-	Load 0x1 into Register B
0x0002	0x30	-	ADD A + B
0x0003	0x00	-	NOP (display result)

Expected Result: LEDs show 0x10 (15 + 1 = 16)

4. Mixed Operation Test

Objective: Verify seamless switching between computation and display

Test Program:

0x0000:	0x85	; Display 0x5 on LEDs
0x0001:	0x15	; Load 0x5 into Register A
0x0002:	0x23	; Load 0x3 into Register B
0x0003:	0x30	; ADD operation
0x0004:	0x00	; Display result (should show 0x8)
0x0005:	0x8C	; Display 0xC on LEDs

Expected Sequence: 0x5 → (computation) → 0x8 → 0xC

Logic Analyzer Validation

Channel Assignment

Channels	Signal	Purpose
0-15	Address bus	Program counter tracking
16-23	Data bus	Instruction and data flow
24-31	Instruction register	Decode verification
32-35	Opcode decode outputs	Control signal verification
36-43	ALU inputs (Reg A, Reg B)	Data routing validation
44-51	ALU output	Computation verification
52-59	LED outputs	Final result display
60	System clock	Timing reference
61-63	Control enables	Data path control

Timing Analysis

- **Instruction Fetch Cycle:** Address stable → Data valid → Register latch
- **Decode Delay:** Instruction register → Control signals active
- **Data Routing Delay:** Control active → Data path established
- **ALU Computation:** Data stable → Result valid
- **Output Update:** Result valid → LED display change

Success Criteria

- ☐ Instructions fetch correctly from sequential memory addresses
- ☐ Instruction decode generates proper control signals
- ☐ Data routing prevents bus contention in all modes
- ☐ ALU operations execute with correct results
- ☐ Display operations show immediate data correctly
- ☐ Mixed programs execute without errors
- ☐ Timing margins adequate for reliable operation
- ☐ No glitches or race conditions observed

Performance Validation

Execution Speed Test

- Measure instruction cycle time (fetch → decode → execute → display)
- Determine maximum reliable clock frequency
- Verify timing margins under worst-case conditions

Program Complexity Test

Complex Test Program:

- Load two 8-bit numbers
- Perform addition
- Check for overflow (future enhancement)
- Display result
- Perform subtraction
- Display result
- Loop back to start (manual reset for now)

Troubleshooting Guide

Issue: Wrong Instruction Decode

- Verify MC74F521N comparator wiring
- Check instruction register data validity
- Confirm comparator reference values

Issue: Bus Contention

- Verify control signal timing
- Check buffer enable logic
- Ensure proper signal isolation

Issue: ALU Results Incorrect

- Verify register loading from instruction data
- Check ALU function select logic
- Confirm data path integrity

Issue: Display Corruption

- Check LED buffer enable signals
- Verify data path switching timing
- Confirm output latch timing

Integration Notes for Phase 4

- Document working instruction cycle timing
- Identify control signals needed for jump operations
- Note memory access patterns for optimization
- Record maximum program complexity achieved
- Plan instruction set extensions

Documentation Deliverables

- Complete instruction set documentation
- Timing diagrams for all instruction types
- Test program examples with expected results
- Logic analyzer captures showing correct operation
- Integration schematic with control logic detail

- Performance measurement data